*In God we trust*

***Introduction to the xv6 Operating System Kernel***

***Students:*** *Mahdis Mirzaei, Bahareh Einollahi, Rayhaneh Einollahi*

*Operating Systems Laboratory – Project 1*

**Github Repository** : https://github.com/Rayhaneh-Einollahi/OS

*School of Electrical and Computer Engineering - University of Tehran*

*October 17, 2025*

# *Questions*

**1.Name the three main tasks of an operating system.**

- To share a computer among multiple programs and provide more useful services than hardware alone.
- To manage and abstract low-level hardware.
- To share hardware among multiple programs so they can run (or appear to run) at the same time.
- To provide controlled ways for programs to interact and share data or work together.

---

**2.Is the presence of an operating system required in all devices? Why? In what situations is it necessary?**

- No, not every device needs an operating system. An OS is necessary when we want to run **multiple programs**, **share hardware**, **provide services** beyond what hardware offers, or allow programs to **interact**. Simple devices that run a single task don't require one.

---

**3.What is the architecture of the xv6 operating system? Provide reasons for your answer.**

- The architecture of xv6 is a **kernel-based** structure. It has two main parts: user space and kernel space. Each program, called a process, runs in user space with its own memory for instructions, data, and stack. When a process needs a service, it calls a system call, which switches execution to the kernel. The kernel runs with hardware privileges, manages resources, and uses the CPU's protection mechanisms so that each process can only access its own memory.

---

**4.Is xv6 a single-tasking or multi-tasking system?**

- xv6 is a **multi-tasking system**. It can time-share processes, meaning it **switches** the CPU between **multiple processes** so that they run, or appear to run, at the same time.

---

**5.As you know, multitasking is a generalization of multiprogramming. What is the difference between a program and a process?**

- A **program** is just a file containing instructions and data stored on disk.
- A **process** is a running instance of a program — it has its own memory (instructions, data, stack) and CPU state. Changing variables in one process does not affect another, even if they come from the same program.

---

**6.What parts make up the structure of a process in the xv6 operating system? How does the system allocate the processor among processes?**

A process in xv6 contains:

- **User-space memory:** instructions, data, and stack
- **Per-process kernel state:** process ID (pid), CPU registers, and other control information

The system allocates the processor by **time-sharing** — it switches the CPU between multiple processes, saving and restoring their registers so that each process gets a chance to run.

---

**7.What is the concept of a file descriptor in UNIX-based operating systems? How does the pipe work in xv6, and what is its typical use?**

**File descriptor** : A file descriptor is a small integer that refers to an object managed by the kernel, such as a file, device, or pipe. It allows a process to perform read and write operations without worrying about the details of the underlying resource.

**Pipe** : A pipe in xv6 is a kernel buffer with two file descriptors: one for reading and one for writing. When one process writes data into the pipe, another process can read it from the other end. Pipes are typically used for **communication between processes (IPC)** and for building **pipelines** (e.g., connecting the output of one program to the input of another).

---

### 8.What do the system calls exec and fork do in xv6? From a design perspective, what is the advantage of not combining these two?

- **fork()** creates a new process (child) that is an exact copy of the parent process, including its memory and file descriptors.
- **exec()** replaces the current process's memory with a new program loaded from a file and starts executing it.

The advantage of keeping them separate is that it gives flexibility — for example, the shell can **fork** a child process, change its file descriptors or environment, and then use **exec** to run a new program. If they were combined, this kind of setup (like I/O redirection or pipelines) would be much more difficult.

---

### 9.Run the make -n command. Which command creates the final kernel file?

When you run the **make** command in the xv6 source directory, it compiles all source files into object files and then uses the linker command **ld** to create the final kernel binary.

```
mhieir@Linux:~/Desktop/OS$ make -n
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb -m32 -fno-omit-frame-pointer -fno-stack-protector -fn
o-pie -no-pie   -c -o console.o console.c
ld -m    elf_i386 -T kernel.ld -o kernel entry.o bio.o console.o exec.o file.o fs.o ide.o ioapic.o kalloc.o kbd.o lapic.o log.o
main.o mp.o picirq.o pipe.o proc.o sleeplock.o spinlock.o string.o swtch.o syscall.o sysfile.o sysproc.o trapasm.o trap.o uart.o
 vectors.o vm.o  -b binary initcode entryother
objdump -S kernel > kernel.asm
objdump -t kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > kernel.sym
dd if=/dev/zero of=xv6.img count=10000
dd if=bootblock of=xv6.img conv=notrunc
dd if=kernel of=xv6.img seek=1 conv=notrunc
```

---

**Boot message**

```
SeaBIOS (version 1.16.2-debian-1.16.2-1)


iPXE (http://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1EFCF250+1EF0F250 CA00



Booting from Hard Disk...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap star
t 58

 -------------Group Members-------------
Rayhaneh Einollahi, Bahareh Einollahi, Mahdis Mirzaei


init: starting sh
$
```

**New feature**

- The most recently used command among the previously executed ones is highlighted in a lighter color, as illustrated in the screenshot below.




- The exact current time is displayed at the end of each line, as illustrated in the screenshot below




---

**10.In the Makefile, variables named UPROGS and ULIB are defined. What are their uses?**

- **UPROGS**: This variable lists all the user programs (binaries) that will be built and included in the xv6 file system image. These are the user-space commands (like sh, echo, ls, etc.) that will be compiled and then copied into the xv6 disk image so they can be executed inside the OS.
- **ULIB**: This variable lists the user-space library source files that are compiled and linked with each user program in UPROGS. These libraries include essential system call wrappers and helper functions used by user programs to interact with the kernel

---

**11.Why is there no user-level program for the `cd` command in xv6 (unlike `ls` or `cat`)? Where is it executed, and why is it different?**

The **cd** command is not implemented as a separate user program like **ls** or **cat**. Instead, it is built directly into the shell and runs inside the shell process itself. The reason is that cd must change the **current working directory of the shell process**. If it were implemented as a normal user program, the shell would use **fork()** to create a child process, and cd would only change the working directory of that child. After the child finishes and exits, the parent shell's working directory would remain unchanged — making cd useless. Because of this, cd is treated differently and is executed inside the shell so that it changes the current directory of the shell process itself.

---

**12.In xv6, what file's content is stored in the first bootable sector of the disk?**

The first bootable sector of the xv6 disk contains the contents of the **bootblock** file, which is created from the bootloader source code. This bootblock is responsible for loading the kernel into memory and starting the operating system.

**13.Compiled programs are stored as binary files. The boot program is also a binary file. What is the type of this binary file?**

- The bootloader binary (bootblock) is a raw binary image — not an ELF file. It is a simple, flat binary that can be directly loaded into memory by the BIOS without any additional parsing or headers.

**Why is this type used?**

- The BIOS expects the first 512 bytes of a bootable disk (the boot sector) to be a raw binary containing executable machine code. It loads this sector directly into memory (usually at address 0x7C00) and begins execution. Using a raw binary ensures compatibility with the BIOS boot process.

**What is the difference between this binary file and other xv6 binary files?**

- **Bootloader (bootblock)** : a small raw binary with no headers, loaded directly by the BIOS.
- **Other xv6 programs** : typically ELF files that include headers, metadata, and multiple sections. These files are loaded by the bootloader or operating system, not directly by BIOS.

**Convert this file into human-readable assembly code.**

```
mhieir@Linux:~/Desktop/OS$ objdump -D -b binary -mi386 -Maddr16,data16 bootblock > bootblock.asm
```

- disassembled the bootblock binary into a human-readable assembly file called bootblock.asm.

```
mhieir@Linux:~/Desktop/OS$ cat bootblock.asm

bootblock:     file format binary


Disassembly of section .data:

00000000 <.data>:
   0:   fa                      cli
   1:   31 c0                   xor    %ax,%ax
   3:   8e d8                   mov    %ax,%ds
   5:   8e c0                   mov    %ax,%es
   7:   8e d0                   mov    %ax,%ss
   9:   e4 64                   in     $0x64,%al
   b:   a8 02                   test   $0x2,%al
   d:   75 fa                   jne    0x9
   f:   b0 d1                   mov    $0xd1,%al
  11:   e6 64                   out    %al,$0x64
  13:   e4 64                   in     $0x64,%al
  15:   a8 02                   test   $0x2,%al
  17:   75 fa                   jne    0x13
  19:   b0 df                   mov    $0xdf,%al
  1b:   e6 60                   out    %al,$0x60
  1d:   0f 01 16 78 7c          lgdtw  0x7c78
  22:   0f 20 c0                mov    %cr0,%eax
```

**14.What is the purpose of the `objcopy` command in the xv6 build process?**

The objcopy tool is used to convert **object files or ELF** executables into a **raw binary** format. In xv6, it's typically used during the bootloader build process to transform the compiled bootloader (which might include ELF headers and

metadata) into a pure flat binary image that the BIOS can load directly from disk.. objcopy strips out unnecessary headers and sections, leaving only the instructions and data that should be executed.

---

**15. In xv6, we see that the system boot process is handled by the files `bootasm.S` and `bootmain.c`. Why isn't only C code used?**

We cannot write the entire bootloader in C because some of the **earliest steps** of the boot process **require direct hardware control and CPU setup**, which C alone cannot perform. These steps must be done in assembly language (bootasm.S) before switching to C (bootmain.c).

- CPU starts in real mode
- Low-level initialization is needed first
- C code is used after initialization

---

**16. Name one general-purpose register, one segment register, one status register, and one control register in the x86 architecture, and briefly explain their roles.**

- General-purpose register (**EAX**) : This register is commonly used as the accumulator in arithmetic, logic, and data movement operations. For example : results of addition or multiplication are often stored in EAX.
- Segment register (**CS - code segment**) : CS holds the base address of the current code segment in memory. It tells the CPU where to fetch the next instructions to execute.
- Status register (**EFLAGS**) : The EFLAGS register contains status and control flags that reflect the outcome of CPU operations. For example, the Zero Flag (ZF) is set if the result of an operation is zero.
- Control register (**CR3**) : CR3 holds the physical address of the page directory used by the Memory Management Unit (MMU) for virtual memory translation.

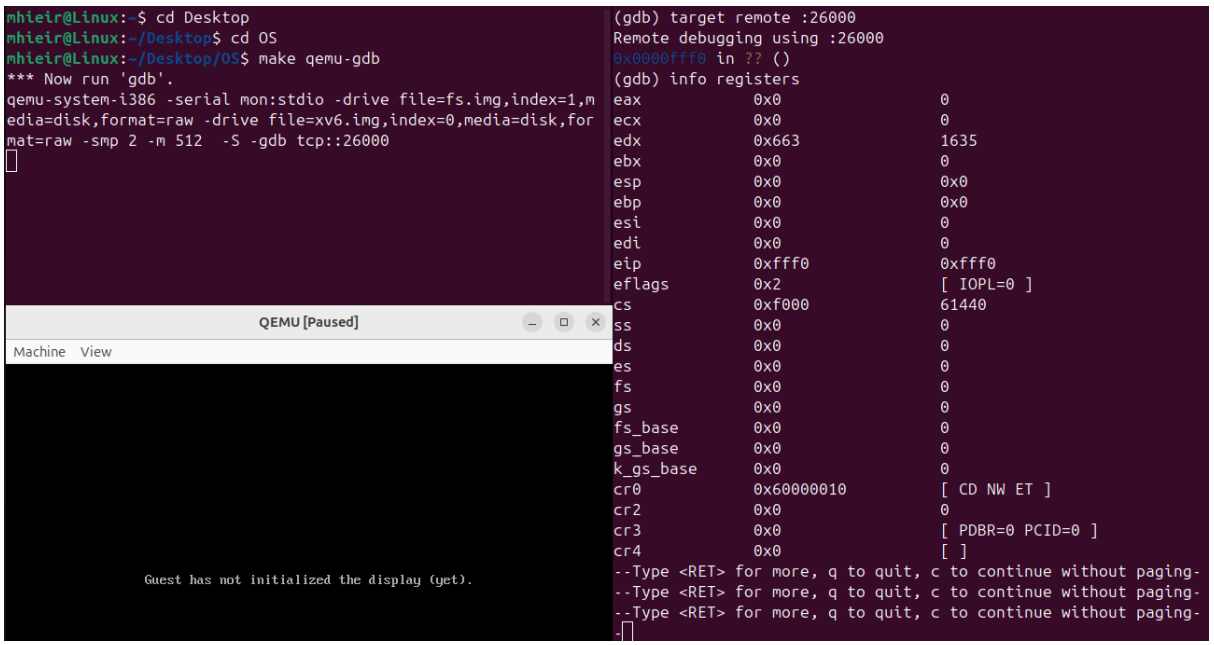
---

**16. Name one general-purpose register, one segment register, one status register, and one control register in the x86 architecture, and briefly explain their roles.**

- General-purpose register (**EAX**) : This register is commonly used as the accumulator in arithmetic, logic, and data movement operations. For example : results of addition or multiplication are often stored in EAX.
- Segment register (**CS - code segment**) : CS holds the base address of the current code segment in memory. It tells the CPU where to fetch the next instructions to execute.
- Status register (**EFLAGS**) : The EFLAGS register contains status and control flags that reflect the outcome of CPU operations. For example, the Zero Flag (ZF) is set if the result of an operation is zero.
- Control register (**CR3**) : CR3 holds the physical address of the page directory used by the Memory Management Unit (MMU) for virtual memory translation.

---

**Info register - gdb**




---

**17. x86 processors have several modes. At boot, they start in real mode (the mode MS-DOS ran in). Why?**

When an x86 processor powers on, it enters real mode to ensure backward compatibility with the original 8086 processors and early software (such as BIOS and MS-DOS). This mode is very simple, it allows the CPU to immediately begin executing basic code (like the BIOS or bootloader) without needing any prior setup.

**State one main drawback of this mode.**

A major limitation of real mode is that it **lacks modern protection** and memory features.

- It can only address 1 MB of memory
- It has no memory protection, so programs can overwrite each other's memory.
- There is no privilege separation between the operating system and user code.

**Are modes in other processors like ARM or RISC-V designed the same way? Explain.**

Other architectures like **ARM** and **RISC-V** do not start in a legacy real mode. They boot directly into a privileged mode that already supports full memory access and protection. These architectures were designed more recently, so they skip the legacy real mode entirely.

---

**18. Another important mode is the protected mode. What is the main function of this mode?**

The main purpose of protected mode is to enable advanced features required by modern operating systems, such as Memory protection, Privilege levels, Virtual memory and paging, Multitasking.

**When do processors enter this mode?**

The processor enters protected mode when the PE (Protection Enable) bit in the CR0 control register is set, which is usually done by the bootloader early in the operating system initialization process.

---

**19. The `bootmain.c` code loads the kernel starting from one sector after the boot sector and places it at address `0x100000`. Why is this address chosen?**

The address 0x100000 (1 MB) is a standard starting point for the kernel in x86 systems. This location is chosen because it is **above the memory used by BIOS,** real-mode interrupts, and the bootloader, but still within the physical memory accessible early in the boot process. It ensures the kernel has a large, continuous memory space available for itself and for paging structures.

**Why doesn't it start from address 0?**

The lower part of memory (below 1 MB) is already reserved for important system components used during boot, so loading the kernel at address 0 would **overwrite critical structures** and cause the system to crash.

---

**Difference breakpoint**

```
(gdb) break main
Breakpoint 1 at 0x80103e80: file main.c, line 20.
(gdb) break userinit
Breakpoint 2 at 0x801047e0: file proc.c, line 126.
(gdb) break exec
Breakpoint 3 at 0x80101900: file exec.c, line 20.
(gdb) break fork
Breakpoint 4 at 0x80104950: file proc.c, line 61.
(gdb) break scheduler
Breakpoint 5 at 0x80104a60: file proc.c, line 326.
(gdb) break proc.c:200
Breakpoint 6 at 0x801049a2: file proc.c, line 200.
(gdb) break *0x80100000
Breakpoint 7 at 0x80100000
(gdb) continue
Continuing.

Thread 1 hit Breakpoint 1, main () at main.c:20
20        kinit1(end, P2V(4*1024*1024)); // phys page allocator
(gdb)
```

---

**20. Which command is used to view breakpoints in GDB?**

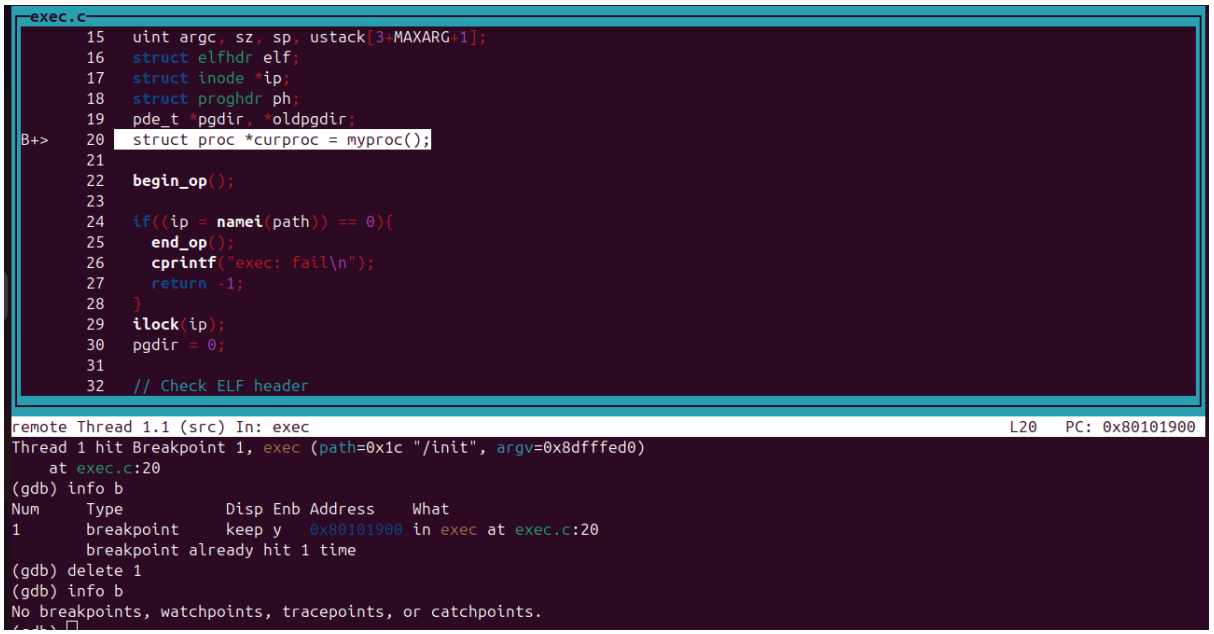
To display all the breakpoints that are currently set in GDB, use the **info b.** It will list all breakpoints with their numbers, locations (file and line or function), and whether they are enabled or disabled.

---

**21. Which command is used to delete a breakpoint and how?**

To delete a specific breakpoint, use the **delete command followed by the breakpoint number** (from the list shown by info b) and also to delete all breakpoints at once just use **delete.**

We executed this command in our GDB session as well, and the result (showing deleted breakpoints) is shown in the picture on the next page.

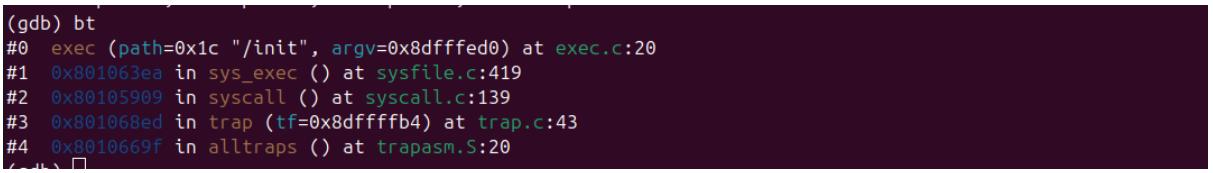



---

**22. Run the following command. What does its output show?**

```
$ bt
```

The **bt** command in GDB stands for **backtrace**. When the program stops (for example, at a breakpoint), this command displays the call stack, a list of all the functions that have been called up to the current point of execution.

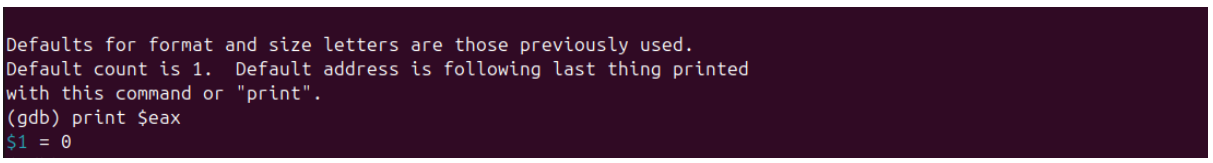



---

**23. Explain two differences between the `x` and `print` commands. How can the contents of a specific register be printed?**

- **Purpose and usage**: **print** is used to evaluate and display the value of variables or expressions in a human-readable format. **x** is used to examine raw memory content at a specific address.
- **Input type and output format: print** works with C-like expressions (variables, structs, arrays, arithmetic). **x** requires a memory address and shows the raw memory content in a specified format (hex, decimal, chars).

To print the content of a specific register, we use the **print** command followed by the register name with a $ sign.




---

**24. Which command is used to display the status of registers?**

```
(gdb) info registers
eax            0x0                0
ecx            0x0                0
edx            0x0                0
ebx            0x1                1
esp            0x8dfffeac         0x8dfffeac
ebp            0x8dffff68         0x8dffff68
esi            0x8dfffed0         -1912602928
edi            0x4                4
eip            0x80101900         0x80101900 <exec>
eflags         0x296              [ IOPL=0 IF SF AF PF ]
```

**How about local variables?**

```
(gdb) info locals
s = <optimized out>
last = <optimized out>
i = <optimized out>
off = <optimized out>
argc = <optimized out>
sz = <optimized out>
sp = <optimized out>
ustack = {2382364176, 2148550465, 2148614240, 2148612832, 2382364192, 2148550465, 2148614292, 2148612416, 2382364208,
  2148550465, 2148614292, 2148612416, 2382364224, 2148553509, 2382364300, 2148612416, 2382364240, 2148550604, 2148622272,
  2382364300, 2382364288, 2148558819, 16843009, 16843009, 16843009, 16843009, 16843009, 2148550465, 2148614292, 2382364368,
  2382364296, 2148553509, 40, 2382364368, 2382364312, 2148550604}
elf = {magic = 16843009, elf = '\001' <repeats 12 times>, type = 257, machine = 257, version = 16843009, entry = 2148550465,
  phoff = 2148612609, shoff = 2148614240, flags = 2382364160, ehsize = 21264, phentsize = 32784, phnum = 15072,
  shentsize = 32785, shnum = 16480, shstrndx = 32785}
ip = <optimized out>
ph = {type = 16843009, off = 16843009, vaddr = 16843009, paddr = 16843009, filesz = 16843009, memsz = 16843009,
  flags = 16843009, align = 16843009}
pgdir = <optimized out>
oldpgdir = <optimized out>
curproc = <optimized out>
(gdb)
```

**Explain the meaning of the `edi` and `esi` registers in the x86 architecture.**

- **EDI** (Destination Index) : Used as a destination pointer in string and memory operations. For example, instructions like movsb, stosd, or rep movsb use EDI to point to the memory location where data will be written.
- **ESI** (Source Index) : Used as a source pointer in string and memory operations. For example, it points to the memory location from which data will be read during block data transfer instructions.

---

**25. Using GDB, explain the structure `struct input`, its inner variables and their roles. Also explain how and when these variables change (for example, when `input.e` changes and what value it takes).**

In xv6, the struct input is a data structure used to handle keyboard input from the console. It stores user-typed characters and manages how they are read and delivered to the shell or other programs. It essentially acts as a buffer between the hardware keyboard and the software that reads user input.

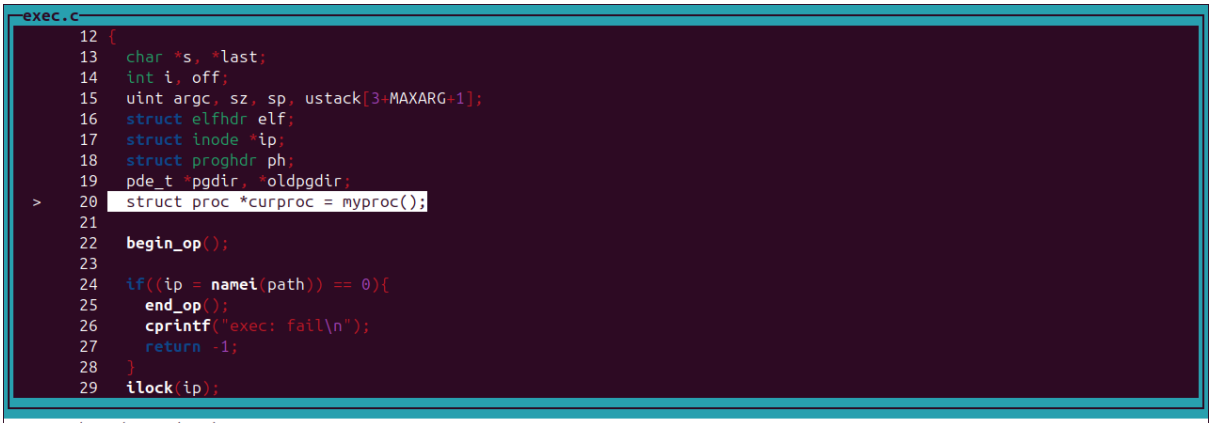
```
type = struct {
    char buf[128];
    uint r;
    uint w;
    uint e;
    uint cur;
    uint rtil;
}
(gdb) print input
$2 = {buf = '\000' <repeats 127 times>, r = 0, w = 0, e = 0, cur = 0, rtil = 0}
(gdb)
```

- **input.w (write index)** : Points to the position after the \n (newline) character. It indicates where the kernel will write the next input once the user presses Enter.
- **input.r (read index)** : Points to the first character of the current line. It shows where the next character will be read by the program.
- **input.rtil (read-until)** : Used internally when handling special input like a tab (\t). During tab completion, r reads input up to rtil. Once the tab handling is complete, r returns to the beginning of the line.
- **input.cur (cursor)** : Represents the current cursor position while the user is typing or editing input.
- **input.e (edit index)** : Points to the end of the current line. It marks the position just after the last typed character.

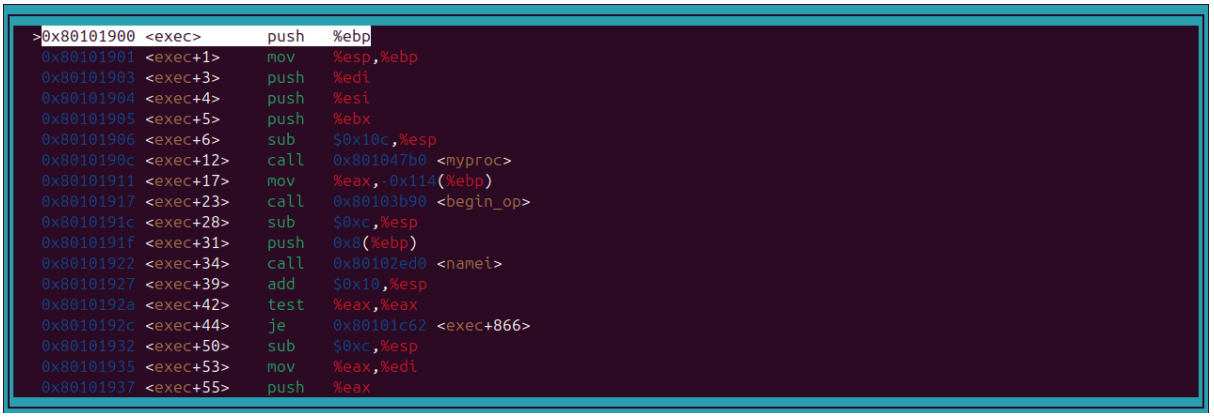
---

### 26. What is the output of the layout src and layout asm commands in TUI?

- **layout src** : Shows the source code view in GDB's TUI mode. It displays your program's C source lines with the current execution line highlighted.




- **layout asm:** : Shows the assembly view instead of C code. It displays the machine instructions being executed and highlights the current instruction.




---

### 27. Which commands are used to move between functions in the current call chain (at a breakpoint)?

When the program stops at a breakpoint, you can use the following commands in GDB to navigate between functions in the current call stack (backtrace):

- **up** : Moves one level up in the call chain (to the function that called the current one).
- **down** : Moves one level down in the call chain (back to the function that was called).
- **frame n** : Jumps directly to a specific frame number in the call stack shown by bt.

